Taller 4

Maria Lucía Castillo García

Índice de contenido

Preguntas de reflexión....1
Parte 3: Investigación de comandos....2
Reto integrador: informe de diagnóstico....3
Evidencia de ejecución de cada actividad....4
Parte 1: Funciones....4
Parte 2: Pipes....4
Parte 4: Proceso consumiendo recursos....5
Parte 5: Logs JSON....7
Reto integrador....8

Preguntas de reflexión

1) ¿Qué ventajas ofrecen los pipes?

- Los pipes permiten conectar varios comandos para que la salida de uno se convierta en la entrada del siguiente. Esto facilita procesar información paso a paso sin necesidad de crear archivos temporales y hace que los comandos sean más flexibles.

2) ¿Cuándo utilizar grep en lugar de jq?

- Utilizaría `grep` cuando solo necesito buscar palabras o patrones dentro de un archivo de texto. En cambio, `jq` es más útil cuando trabajo con archivos JSON y necesito acceder a campos específicos o filtrar información de manera estructurada.

3) ¿Qué ventajas ofrece awk?

- `awk` permite analizar y procesar datos organizados en columnas de forma muy eficiente. Es útil para filtrar información, realizar cálculos, generar reportes y extraer únicamente los datos que interesan sin tener que escribir programas más complejos.

4) ¿Qué riesgos tiene finalizar procesos incorrectos con kill?

- Si se finaliza un proceso equivocado, se puede interrumpir un servicio importante, perder información que no se haya guardado o afectar el funcionamiento normal del sistema. Por eso es importante verificar cuidadosamente el identificador del proceso antes de usar `kill`.

5) ¿Qué utilidad tiene el análisis de logs en ambientes Cloud?

- El análisis de logs permite monitorear el estado de las aplicaciones y los servicios, detectar errores, identificar problemas de rendimiento y entender qué ocurrió ante una falla. Esto ayuda a resolver incidentes más rápido y a mantener los sistemas disponibles y funcionando correctamente. Los logs son registros que generan los programas, aplicaciones o sistemas para dejar constancia de lo que ocurre mientras están funcionando.

Parte 3: Investigación de comandos

Comando	¿Qué hace?	Sintaxis	Ejemplo	Caso de uso
`grep`	Busca una palabra o texto dentro de un archivo y muestra las líneas donde aparece.	`grep "texto" archivo`	`grep error archivo.txt`	Cuando sé qué palabra estoy buscando y no quiero leer todo el archivo.
`ps`	Me muestra los procesos que se están ejecutando en ese momento. `ps` solo muestra los de mi terminal actual, mientras que `ps aux` muestra TODOS los procesos del sistema, con columnas más detalladas.	`ps [opciones]`	`ps aux`	`ps` cuando estoy trabajando en algo puntual desde mi terminal; `ps aux` cuando necesito diagnosticar algo a nivel de todo el servidor.
`htop`	Muestra los procesos de forma visual e interactiva.	`htop`	`htop`	Cuando quiero ver fácilmente qué está consumiendo CPU o memoria.
`kill`	Finaliza un proceso usando su PID.	`kill PID`	`kill 1860`	Cuando un programa se queda congelado o no responde y necesito cerrarlo.

Comando	¿Qué hace?	Sintaxis	Ejemplo	Caso de uso
`awk`	Permite trabajar con información organizada en columnas y extraer solo lo que me interesa.	`awk '{print $columna}' archivo`	`awk '{print $2}' estudiantes.txt`	Cuando un archivo o salida tiene muchas columnas y solo necesito una o algunas muy específicas.
`jq`	Permite leer y filtrar información de archivos JSON.	`jq '{campo1,campo2}' archivo.json`	`jq '.nombre' datos.json`	Cuando trabajo con APIs o archivos JSON y necesito datos concretos.

Reto integrador: informe de diagnóstico

1. Proceso problemático identificado

- Proceso: `firewalld` (`/usr/bin/python3 -sP /usr/bin/firewalld --nofork --nopid`)
- PID: 749 · Usuario: root · %CPU: 0.0 · %MEM: 2.3

2. Análisis de logs (logs.json)

- Total de registros: 4
- Errores HTTP 500 encontrados: 2
- Servicio más afectado: api y auth (empate, 1 error cada uno)

3. Conclusión

El proceso con mayor consumo de memoria en el servidor fue `firewalld`, con un 2.3% de uso, valor considerado normal para un servicio de seguridad activo. No se detectaron procesos con consumo excesivo o raro de CPU. Por otro lado, el análisis del log del microservicio reveló 2 errores HTTP 500 distribuidos equitativamente entre los servicios api y auth, lo que sugiere revisar la conexión a la base de datos (servicio api) y la validación de tokens de autenticación (servicio auth) como posibles causas del bajo rendimiento reportado por los usuarios.

Evidencia de ejecución de cada actividad

Parte 1: Funciones

Ejemplo

```
casti@MLC MINGW64 ~/taller4_bash
$ nano ejemplo_funcion.sh

casti@MLC MINGW64 ~/taller4_bash
$ chmod +x ejemplo_funcion.sh

casti@MLC MINGW64 ~/taller4_bash
$ ./ejemplo_funcion.sh
Hola, desde Bash :)
```

Función presentarse

```
casti@MLC MINGW64 ~/taller4_bash
$ nano presentarse.sh
```

```
MINGW64:/c/Users/casti/taller4_bash
  GNU nano 8.7
#!/bin/bash

presentarse(){
        echo "Mi nombre es Maria Lucía Castillo García"
        echo "Estudio Ingeniería de Sistemas y Computación"

}


presentarse
presentarse
presentarse
```

```
casti@MLC MINGW64 ~/taller4_bash
$ chmod +x presentarse.sh

casti@MLC MINGW64 ~/taller4_bash
$ ./presentarse.sh
Mi nombre es Maria Lucía Castillo García
Estudio Ingeniería de Sistemas y Computación
Mi nombre es Maria Lucía Castillo García
Estudio Ingeniería de Sistemas y Computación
Mi nombre es Maria Lucía Castillo García
Estudio Ingeniería de Sistemas y Computación
```

Parte 2: Pipes

```
casti@MLC MINGW64 ~/taller4_bash
$ ls | wc -l
2
```

```
casti@MLC MINGW64 ~/taller4_bash
$ cat archivo.txt | grep error
error: no se pudo conectar a la base de datos
error: token inválido

casti@MLC MINGW64 ~/taller4_bash
$
```

ls | wc -l

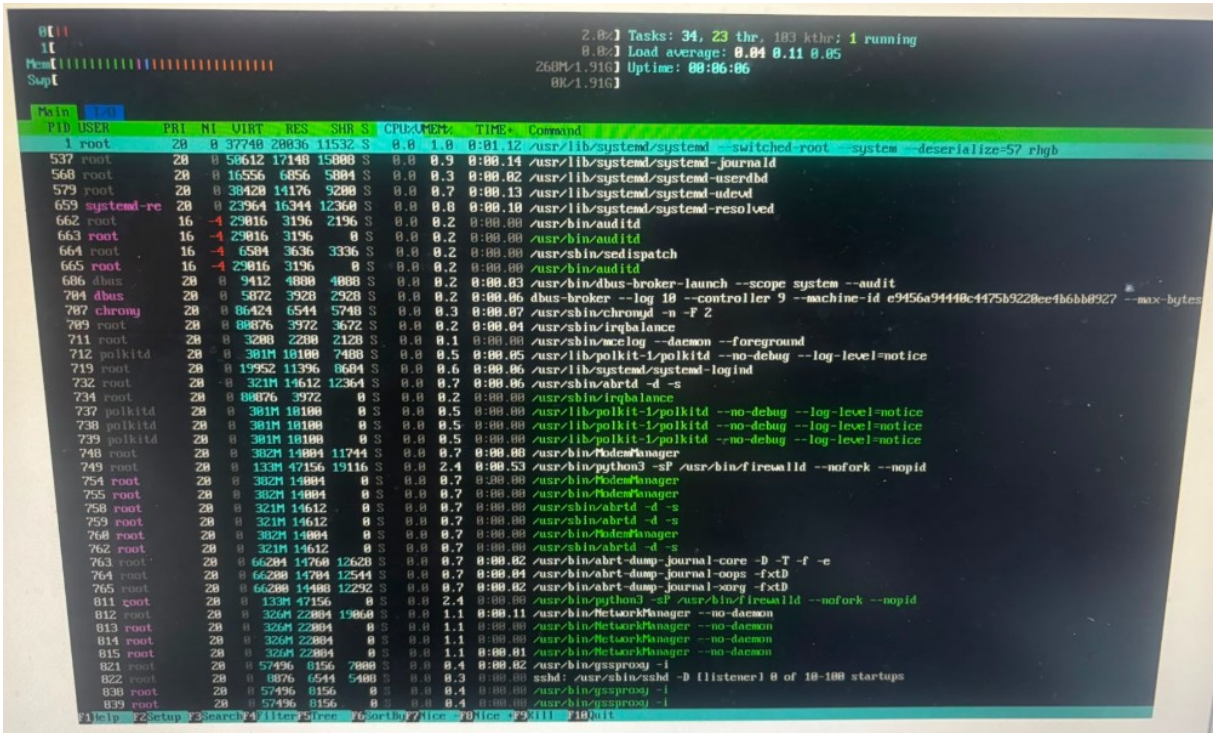
`ls` me obtiene la lista de archivos y carpetas del directorio actual, luego ESA lista se la paso a `wc -l`, para que este último me cuente cuántos elementos hay en total. Es decir, me indica cuántos archivos y carpetas hay en el directorio donde estoy ubicado.

cat archivo.txt | grep error

Primero se lee todo el contenido de `archivo.txt` para mandarle ese contenido a `grep`, y este va a filtrar y mostrar únicamente las líneas donde aparece la palabra "error".

Parte 4: Proceso consumiendo recursos

1. htop



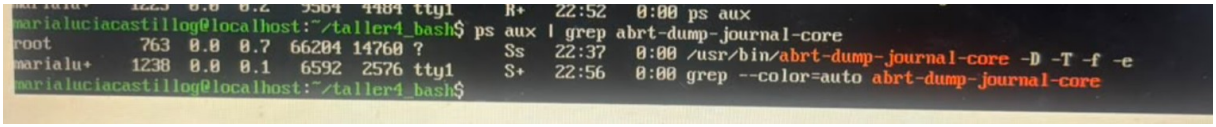
2 y 3. Proceso identificado

PID: 763 · Nombre: `abrt-dump-journal-core` · %CPU: 0.0 · %MEM: 0.7

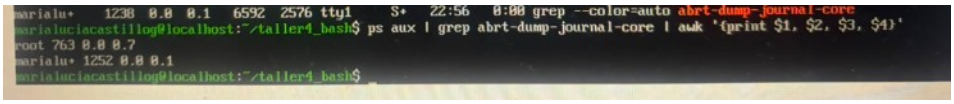
4. ps aux



5. Filtrar con grep



6. awk con usuario, PID, CPU y MEM



7. kill



8. Verificación



Efectivamente ya no aparece el proceso con PID 763.

Parte 5: Logs JSON

1. Líneas con errores

```
casti@MLC MINGW64 ~/taller4_bash
$ nano logs.json

casti@MLC MINGW64 ~/taller4_bash
$ grep error logs.json
{"level":"error","service":"api","status":500,"message":"Database error"}
{"level":"error","service":"auth","status":500,"message":"Token invalid"}

casti@MLC MINGW64 ~/taller4_bash
$
```

2. service, status y message

```
casti@MLC MINGW64 ~/taller4_bash
$ jq '{service,status,message}' logs.json
{
  "service": "api",
  "status": 200,
  "message": "OK"
}
{
  "service": "api",
  "status": 500,
  "message": "Database error"
}
{
  "service": "auth",
  "status": 500,
  "message": "Token invalid"
}
{
  "service": "auth",
  "status": 200,
  "message": "Login success"
}
```

3. Errores HTTP 500

```
casti@MLC MINGW64 ~/taller4_bash
$ jq 'select(.status==500)' logs.json
{
  "level": "error",
  "service": "api",
  "status": 500,
  "message": "Database error"
}
{
  "level": "error",
  "service": "auth",
  "status": 500,
  "message": "Token invalid"
}
```

4. Contar errores

```
casti@MLC MINGW64 ~/taller4_bash
$ jq 'select(.status==500)' logs.json | wc -l
12
```

```
casti@MLC MINGW64 ~/taller4_bash
$ jq -c 'select(.status==500)' logs.json | wc -l
2
```

Con `wc -l` sale 12 porque `jq` muestra cada error en varias líneas. Con `-c` cada error queda en una sola línea y el conteo da 2, que es el número real de errores.

5. Servicio con más errores

```
casti@MLC MINGW64 ~/taller4_bash
$ jq -r 'select(.status==500) | .service' logs.json
api
auth
```

- `select(.status==500)` filtra solo los errores 500.
- `.service` extrae únicamente el campo service de esos resultados.
- `-r` (raw) le quita las comillas que `jq` normalmente le pone al texto.

```
casti@MLC MINGW64 ~/taller4_bash
$ jq -r 'select(.status==500) | .service' logs.json | sort | uniq -c | sort -nr
      1 auth
      1 api

casti@MLC MINGW64 ~/taller4_bash
$
```

Después, `sort` ordena los nombres para agrupar las repeticiones, `uniq -c` cuenta cuántas veces aparece cada servicio y `sort -nr` ordena esos conteos de mayor a menor (`-n` ordena por número y `-r` invierte el orden), para que el servicio con más errores aparezca primero.

Reto integrador

